



山东大学

信息科学与工程学院

2025—2026 学年第一学期

实 验 报 告

课程名称: Java 编程技术

实验名称: 第二次实验

专 业 班 级 通信三班

学 生 学 号

学 生 姓 名

实 验 时 间 2025 年 10 月 5 日

实 验 报 告

【实验目的】

1.继续熟悉 Eclipse 的使用

【实验要求】

一、在 Eclipse 平台中实现下列简单的程序

【实验具体内容】

一、在 Eclipse 平台中实现下列简单的程序

1、用 while 循环语句，计算 1—2000 之间的所有 3 的倍数之和。

(1) 实验源程序

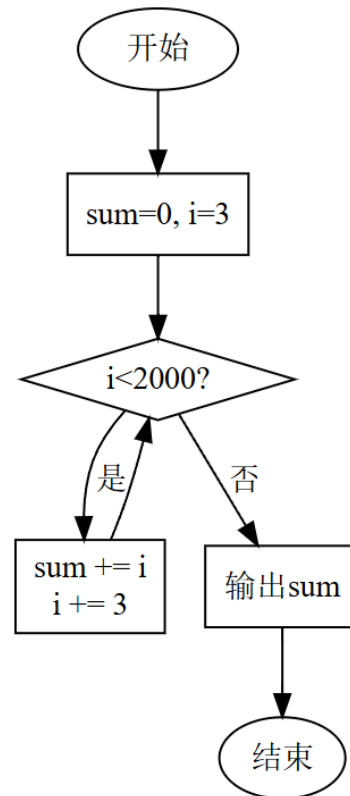
```
package cn;

public class Sum3 {
    public static void main(String[] args) {
        Integer sum= new Integer(0);
        Integer i= new Integer(3);

        while(i<2000) {
            sum += i;
            i = i+3;
        }

        System.out.printf("所有 2000 以内 3 的倍数和为: %d",sum);
    }
}
```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：这是一个非常基础的代码同之前一样，我们同样枚举三的倍数，即枚举时 i 每次递增 3，最后让 sum 累加 i 的值即可，需要注意的只有 sum 开始时注意初始化即可。

1) 编写 java 代码并运行

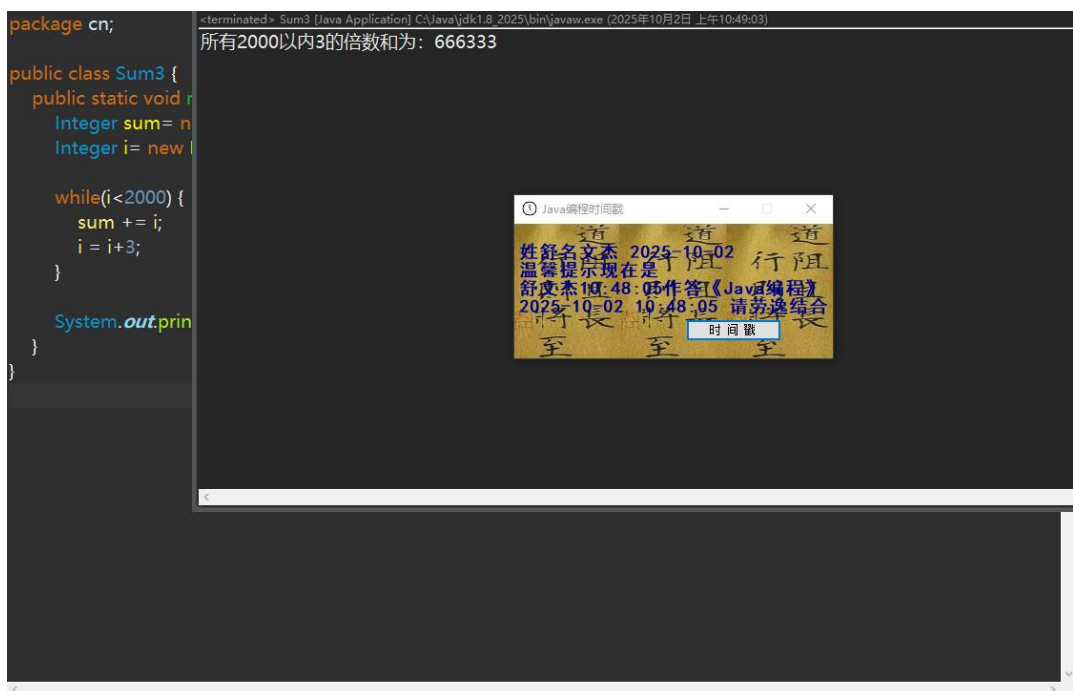


图 1 运行结果截图

可以发现，我们的运行结果符合预期，我们的实验成功。

2、Scanner 类是 java.util 包中的一个常用于控制台的输入的类，当需要使用控制台输入时，可调用该类完成程序交互，利用 Scanner 类和 switch 语句实现输入某年某月，判断某年（考虑闰年）的某个月份有几天的程序。

(1) 实验源程序

```
package cn;
import java.util.Scanner;

public class MonthDaysCalculator {
    public static void main(String[] args) {
        Scanner scanner = new Scanner(System.in);
        System.out.print("请输入年份: ");
        int year = scanner.nextInt();

        System.out.print("请输入月份: ");
        int month = scanner.nextInt();

        int days = getMonthdays(year, month);
        System.out.printf("这个月有%d天 ", days);
        scanner.close();
    }

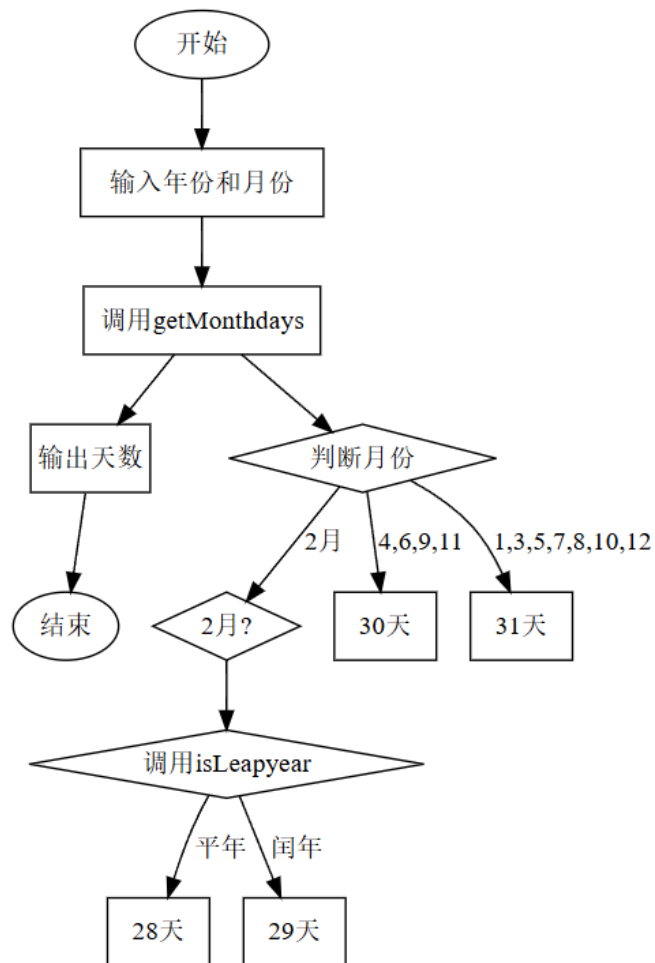
    static int getMonthdays(int year, int month) {
        int days = 0;
        switch(month){
            case 1: days = 31; break;
            case 2: if(isLeapyear(year)) {
                days = 28;
            }
            else {
                days = 29;
            }
            break;
            case 3: days = 31; break;
            case 4: days = 30; break;
            case 5: days = 31; break;
            case 6: days = 30; break;
            case 7: days = 31; break;
            case 8: days = 31; break;
            case 9: days = 30; break;
            case 10: days = 31; break;
            case 11: days = 30; break;
        }
    }

    static boolean isLeapyear(int year) {
        return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
    }
}
```

```

        case 12: days =31;break;
    }
    return days;
}
static boolean isLeapyear(int year) {
    return (year % 4 == 0 && year % 100 != 0) || (year % 400 == 0);
}
}
    
```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：对于该代码，我们学习到对于 switch 语句的用法在 Java 中与 c 语言中没什么区别后，因此这也是比较简单的任务。同时，我们利用我们课上学习的 scanner 类的输入方式，这个代码就可以很简单地实现了。

1) 编写 java 代码并运行

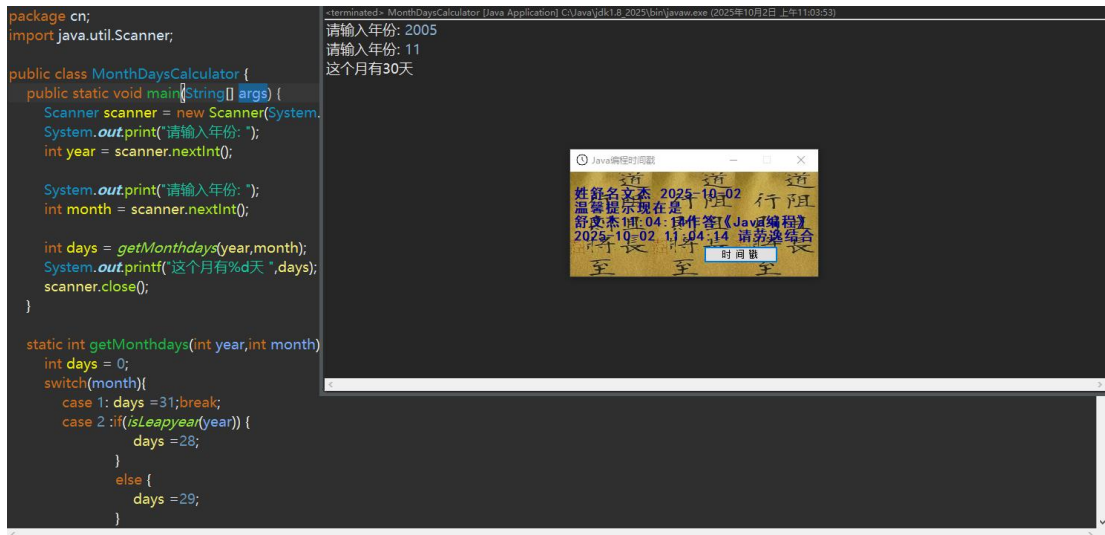


图 2 实验结果截图

可以发现，我们的运行结果符合预期，我们的实验成功。

3、水仙花数是指其个位、十位和百位上三个数的立方之和等于这个数本身。通过循环，判断 100-999 之间所有的数字，符合水仙花数条件的数字。

(1) 实验源程序

```

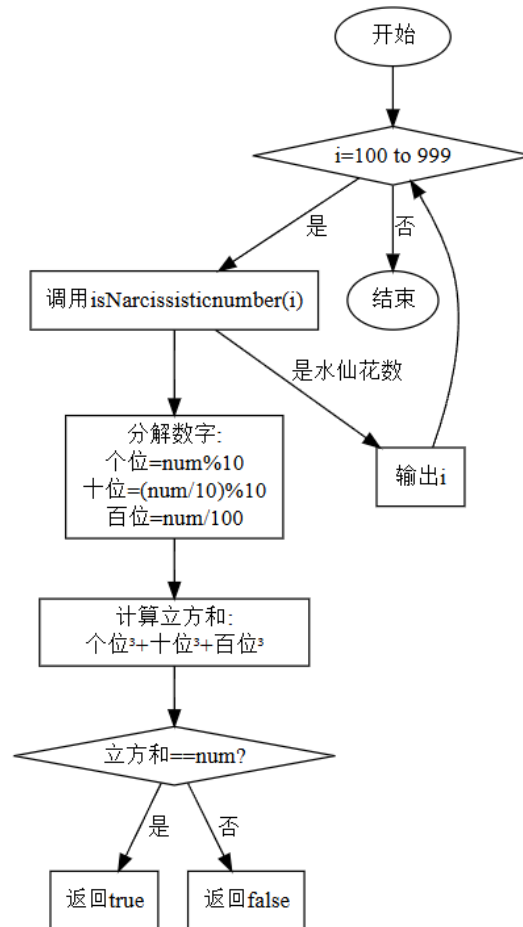
package cn;

public class Narcissisticnumber {
    static boolean isNarcissisticnumber(int num) {
        boolean y = false;
        int one = num%10;
        int ten = (num/10)%10;
        int hundred = num/100;

        if(one*one*one +ten*ten*ten+hundred*hundred*hundred == num) {
            y = true;
        }
        return y;
    }

    public static void main(String[] args) {
        int i;
        for (i =100; i<=999;i+=1) {
            if(isNarcissisticnumber(i)) {
                System.out.printf("%d ",i);
            }
        }
    }
}
    
```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：对于水仙花数，由于我们只用判断三位数，因此我们可以直接提取个，十，百位，利用定义判断是否为水仙花数，同样的枚举即可以了。

1) 编写 java 代码并运行

```

package cn;

public class Narcissisticnumber {
    static boolean isNarcissisticnumber(int num) {
        boolean y = false;
        int one = num%10;
        int ten = (num/10)%10;
        int hundred = num/100;

        if(one*one*one + ten*ten*ten + hundred*
            hundred*hundred == num) {
            y = true;
        }
        return y;
    }

    public static void main(String[] args) {
        int i;
        for (i = 100; i <= 999; i++) {
            if(isNarcissisticnumber(i)) {
                System.out.printf("%d ", i);
            }
        }
    }
}
    
```

图 3 实验结果截图

可以发现，我们的运行结果符合预期，我们的实验成功。

3、已知 $XYZ + YZZ = 532$ ，其中 X、Y 和 Z 为数字，编程求出 X、Y 和 Z 的值。

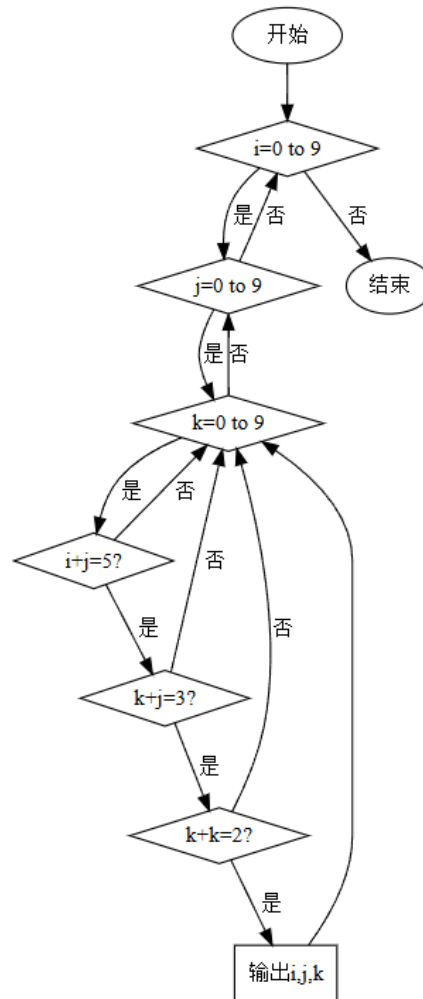
(1) 实验源程序

```
package cn;

public class Formula {
    public static void main(String[] args) {
        int i,j,k;

        for(i = 0; i<=9; i++) {
            for(j = 0; j<=9; j++) {
                for(k = 0; k<= 9; k++) {
                    if(i+j == 5) {
                        if(k+j == 3) {
                            if(k+k == 2) {
                                System.out.printf("x=%d ",i);
                                System.out.printf("y=%d ",j);
                                System.out.printf("z=%d ",k);
                            }
                        }
                    }
                }
            }
        }
    }
}
```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：对于该方程，我们可以将其化简为三个固定的方程，即 $x+y=5$, $y+z=3$, $z+z=2$, 事实上，我们已经可以解出来该方程的解了，但为了暴力一点，我们还是选择了枚举的方法，但是，该方法仍然为我们下一个百钱买百鸡的问题提供了启示。

1) 编写 java 代码并运行

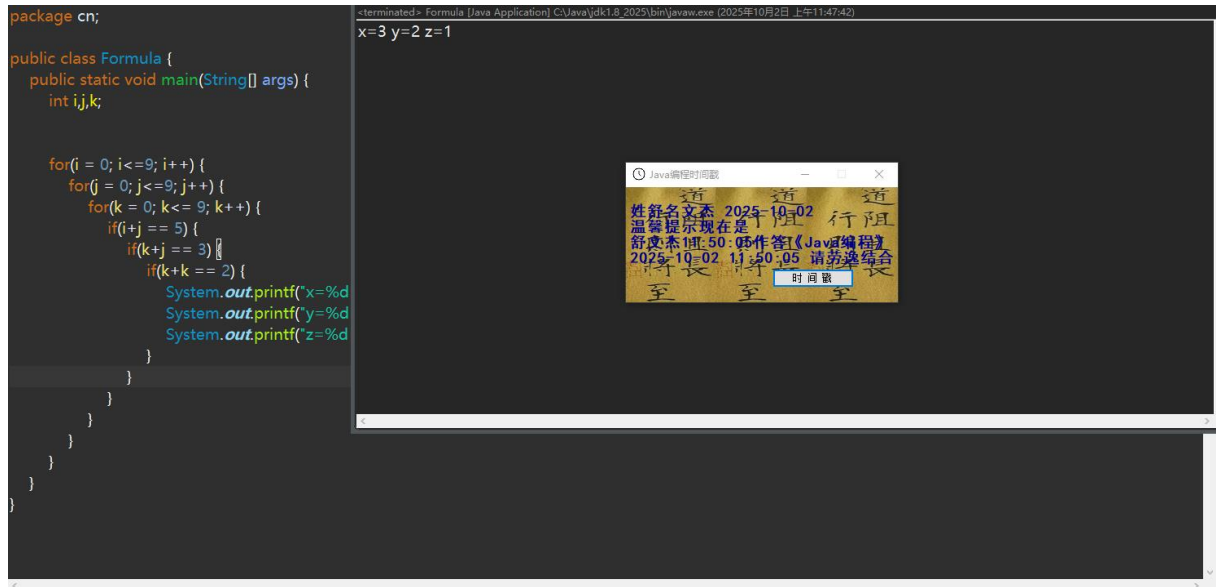


图 4 实验结果截图

可以发现，我们的运行结果符合预期，我们的实验成功。

4、编程实现“百钱买百鸡”问题。母鸡 5 分钱一只，公鸡三分钱一只，小鸡一分钱三只，现在有百钱欲买百鸡，有多少种买法？程序分别用三种方法来写，

(1) 实验源程序

1) 第一种解法（三重循环）

```
package cn;

public class HundredChickensProblem {
    public static void main(String[] args) {
        int i,j,k,n;
        n=0;
        for(i = 0; i<=20; i++) {
            for(j = 0; j<=33; j++) {
                for(k = 0; k<= 100; k++) {
                    if(i*5 + j*3 +k == 100 && i + j +k*3 == 100) {
                        n+=1;
                        System.out.printf("第%d 种买法",n);
                        System.out.printf("母鸡有%d 只 ",i);
                        System.out.printf("公鸡有%d 只 ",j);
                        System.out.printf("小鸡有%d 只 ",k*3);
                        System.out.printf("\n");
                    }
                }
            }
        }
    }
}
```

```

    }
}

```

2) 第二种解法（二重循环）

```

package cn;

public class HundredChickensProblem2 {
    public static void main(String[] args) {
        int n;
        n=0;
        for (int hen = 0; hen <= 20; hen++) {
            for (int cock = 0; cock <= 33; cock++) {
                int chick = 100 - hen - cock;

                // 检查是否满足条件
                if (chick >= 0 && chick%3 == 0 && chick/3 + cock*3 + hen*5 == 100 )
                {
                    n+=1;
                    System.out.printf("第%d 种买法", n);
                    System.out.printf("母鸡: %d 只, 公鸡: %d 只, 小鸡: %d 只\n",
hen, cock, chick*3);
                }
            }
        }
    }
}

```

3) 第三种解法（一重循环）

```

package cn;

public class HundredChickensProblem3 {
    public static void main(String[] args) {
        int n;
        n=0;
        for (int hen = 0; hen <= 20; hen++) {

            if ((100 - 7 * hen) % 4 == 0) {
                int cock = (100 - 7 * hen) / 4;
                int chick = 100 - hen - cock;

                // 检查是否满足所有条件
                if (cock >= 0 && chick >= 0 && chick % 3 == 0) {
                    n+=1;

```

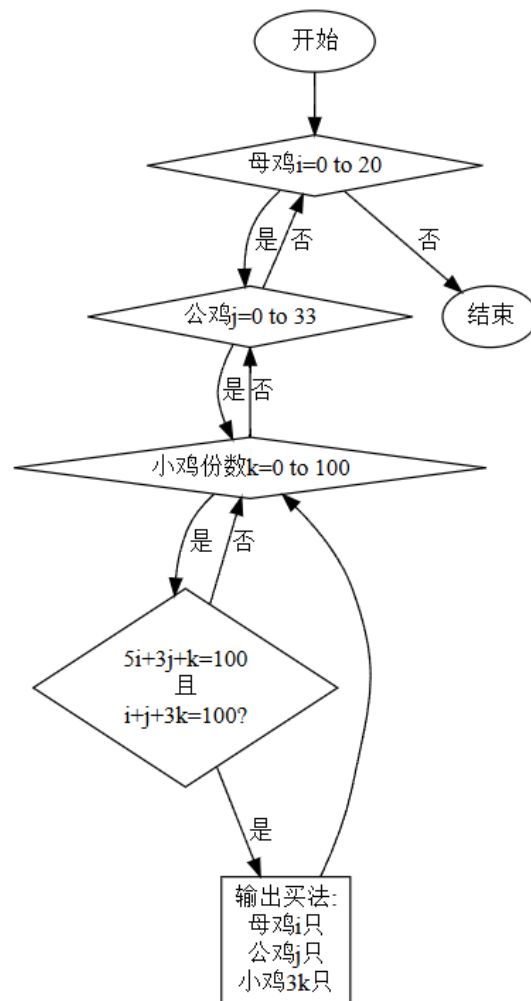
```

        System.out.printf("第%d 种买法\n", n);
        System.out.printf("母鸡: %d 只, 公鸡: %d 只, 小鸡: %d 只\n", hen,
        cock, chick);
    }
}
}
}
}
}
}

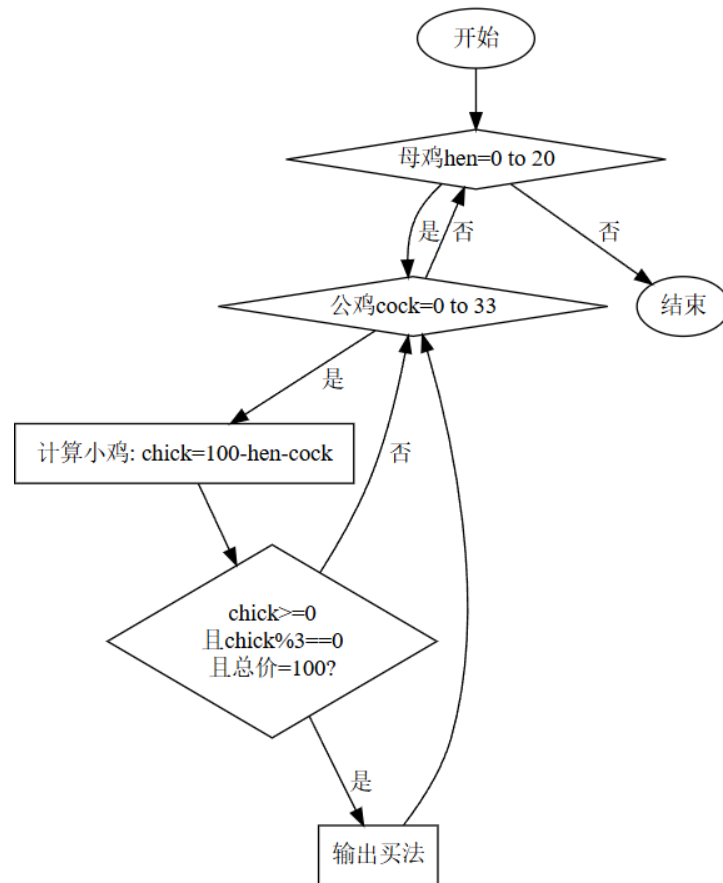
```

(2) 代码流程图

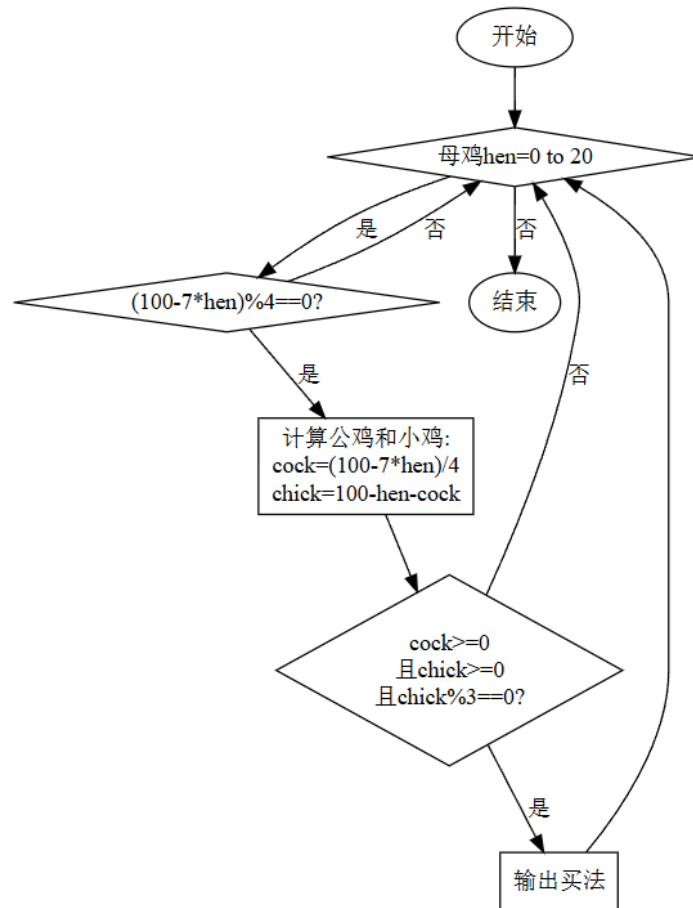
第一种解法（三层循环）



第二种解法（两层循环）



第三种方法（一层循环）



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：

A. 第一种解法：像上一个代码，我们可以列出方程，同样地使用三重循环，暴力枚举三个变量，这样的写法非常简单，但是它的运行效率令人担忧，这时，我们会发现，这三个变量并非没有相互联系，我们可以利用它们关系来化简。

B. 第二种解法：我们可以将其中一个方程带入第二个方程，这样，我们只需要枚举两个变量就可以了，同时，为了减少迭代次数，我们需要将小鸡表示出来，同时我们需要注意，小鸡的数量非负且必须为整数，否则是非法的。

C. 第三种解法：我们可以进一步优化，通过数学推导直接减少需要枚举的变量数量。观察百钱买百鸡问题的数学关系，发现公鸡的数量完全可以被母鸡表示出来，我们完全只需要枚举母鸡的数量，同样保证其他的鸡数量为自然数即可。

1) 编写 java 代码并运行

第一个方法的验证：

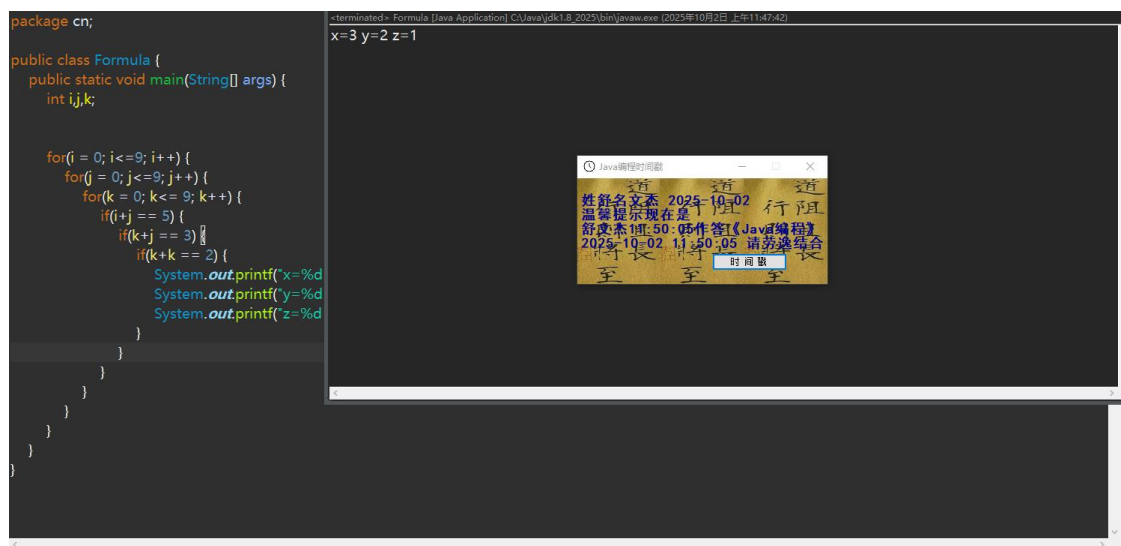


图 5 第一种方法的验证

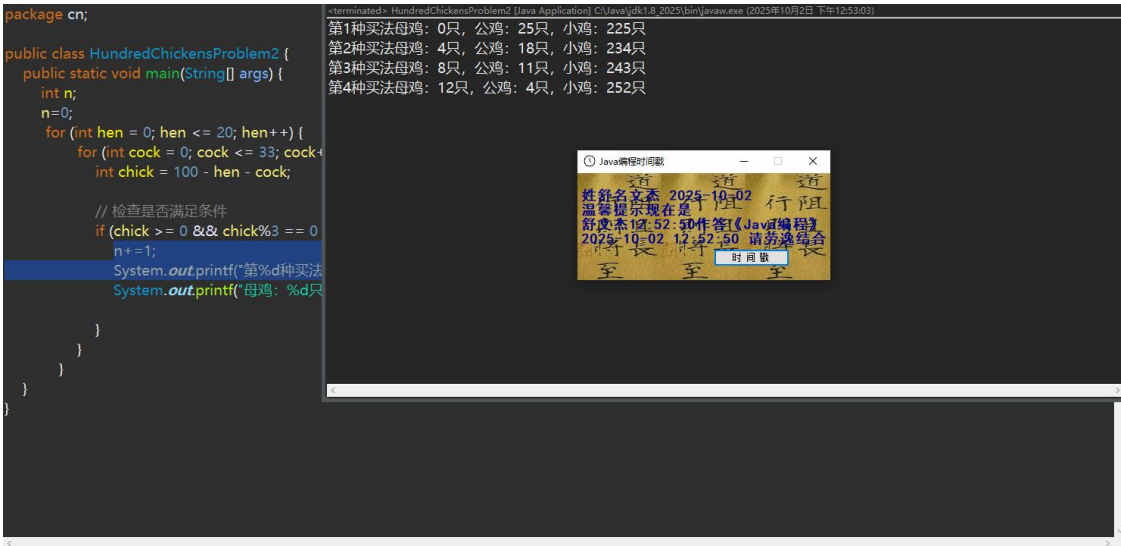


图 6 第二种方法结果截图

```

package cn;

public class HundredChickensProblem3 {
    public static void main(String[] args) {
        int n;
        n=0;
        for (int hen = 0; hen <= 20; hen++) {

            if ((100 - 7 * hen) % 4 == 0) {
                int cock = (100 - 7 * hen) / 4;
                int chick = 100 - hen - cock;

                // 检查是否满足所有条件
                if (cock >= 0 && chick >= 0 && ch
                    n+=1;
                    System.out.printf("第%d种买法\n", n);
                    System.out.printf("母鸡: %d只, 公鸡: %d只, 小鸡: %d只\n", hen, cock, chick);
                }
            }
        }
    }
}
    
```

```

<terminated> HundredChickensProblem3 (Java Application) C:\Java\jdk1.8_2025\bin\javaw.exe (2025年10月2日 下午12:53:59)
第1种买法
母鸡: 0只, 公鸡: 25只, 小鸡: 75只
第2种买法
母鸡: 4只, 公鸡: 18只, 小鸡: 78只
第3种买法
母鸡: 8只, 公鸡: 11只, 小鸡: 81只
第4种买法
母鸡: 12只, 公鸡: 4只, 小鸡: 84只
    
```

图 7 第三种方法结果截图

综上，我们的三个方案的结果得到了一致性，说明我们的代码没有问题，实验成功。

5、用计算机的暴力美学，验证从 $0 \sim 2147483647$ 的整数中下面的定理正确性，当一个整数的各位数字之和能被 9 整除，则该数一定也能被 9 整除。

(1) 实验源程序

初始证明方法：

```

package cn;

public class OptimizeProof {

    public static void main(String[] args) {
        boolean y = true;
        int sum=0,k;
        int mod9Counter = 1;
        long startTime = System.currentTimeMillis();

        for(int i =1;i<214748364;i+=1) {

            if(mod9Counter != 9) {
                mod9Counter += 1;
            }
            else {
                mod9Counter =1;
                continue;
            }
        }
    }
}
    
```

```

        k=i;
        sum =0;
        while (k > 0) {
            int lastTwoDigits = k % 100;
            sum += (lastTwoDigits % 10) + (lastTwoDigits / 10);
            k /= 100;

            if (sum >= 9) {
                sum -= 9;
            }
        }

        if(sum %9 == 0) {
            y = false;
            break;
        }
    }

    long endTime = System.currentTimeMillis();
    System.out.printf("执行时长: %d 毫秒.\n", (endTime - startTime));

    if(y) {
        System.out.print("该命题是正确的");
    }
    else {
        System.out.print("该命题是错误的");
    }
}

```

多线程优化方法

```

package cn;

import java.util.concurrent.*;
import java.util.concurrent.atomic.AtomicBoolean;
import java.util.concurrent.atomic.AtomicInteger;

public class OptimizedDivisibilityValidator {
    static final int MAX_NUM = Integer.MAX_VALUE;
    static final AtomicBoolean found = new AtomicBoolean(false);
    static final AtomicInteger counterexample = new AtomicInteger(-1);

```

```

public static void main(String[] args) throws InterruptedException {
    long startTime = System.currentTimeMillis();
    boolean isValid = optimizedValidate();
    long endTime = System.currentTimeMillis();

    if (isValid) {
        System.out.println("定理验证通过!");
    } else {
        System.out.println("找到反例: " + counterexample.get());
    }

    System.out.println("耗时: " + (endTime - startTime) + "ms");
}

```

```

public static boolean optimizedValidate() throws InterruptedException {
    int threadCount = Runtime.getRuntime().availableProcessors() * 2;
    ExecutorService executor = Executors.newFixedThreadPool(threadCount);
    CountDownLatch latch = new CountDownLatch(threadCount);

    int chunkSize = MAX_NUM / threadCount + 1;

    for (int i = 0; i < threadCount; i++) {
        final int threadId = i;
        executor.submit(() -> {
            try {
                int start = threadId * chunkSize;

                start = Math.max(start, 1);
                int end = Math.min((threadId + 1) * chunkSize - 1, MAX_NUM);

                int mod9Counter = (start % 9 == 0) ? 9 : start % 9;
                validateCheck(start, end, mod9Counter);
            } finally {
                latch.countDown();
            }
        });
    }

    latch.await();
    executor.shutdown();
    return !found.get();
}

```

```

public static void validateCheck(int start, int end, int initialCounter) {
    int mod9Counter = initialCounter;
    for (int num = start; num <= end && !found.get(); num++) {
        if (mod9Counter == 9) {
            mod9Counter = 1;
            continue;
        } else {
            mod9Counter++;
        }

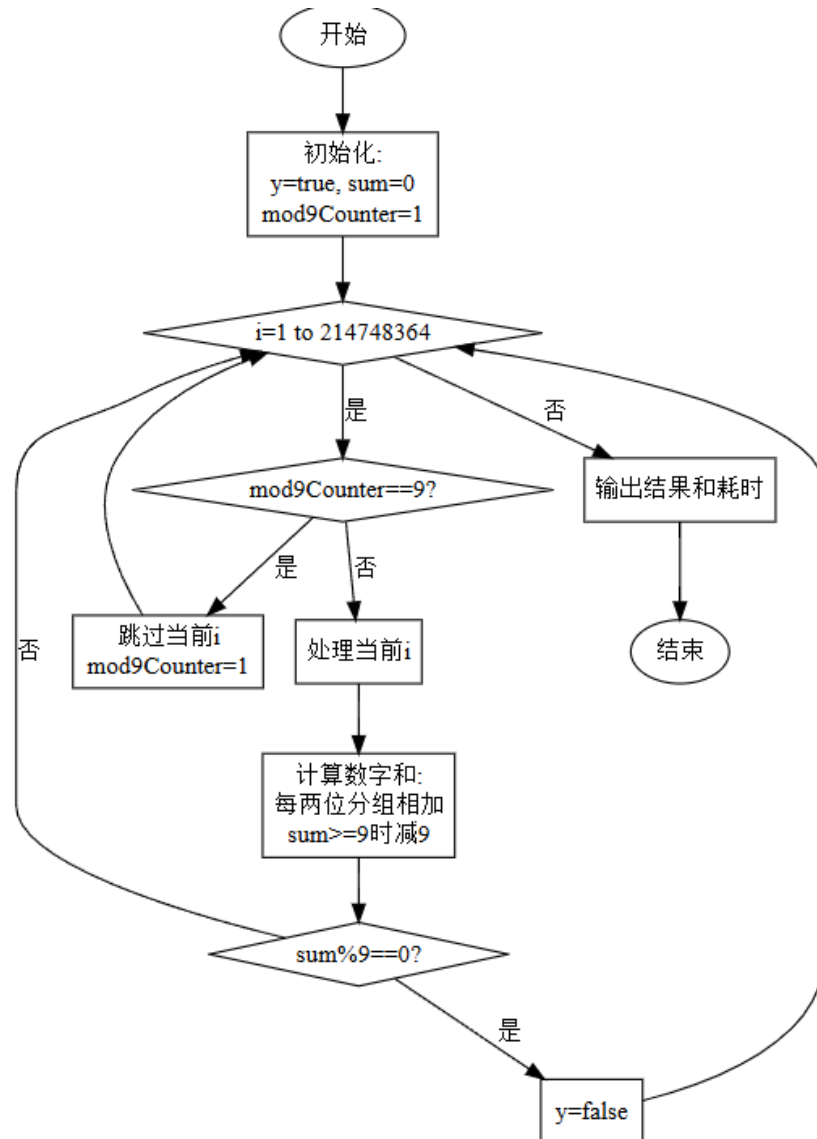
        int sum = 0;
        int k = num;
        do {
            int lastTwoDigits = k % 100;
            sum += (lastTwoDigits % 10) + (lastTwoDigits / 10);
            k /= 100;

            if (sum >= 9) {
                sum -= 9;
            }
        } while (k > 0);

        if (sum == 0) {
            found.set(true);
            counterexample.set(num);
            return;
        }
    }
}

```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：对于该证明优化问题，我们一开始就应该从数学方法的方向进行思考，由于我对于多线程并不太熟悉，因此，我认为我应该先从数学方面进行优化，在初步，我想到的可以优化的主要由两点：1) 对于 9 的倍数，我们可以不验证，这样便可以减少 10%的迭代次数。2) 碰到验证失败的数，即可退出（很可能没有什么用）。

1) 编写 java 代码并运行

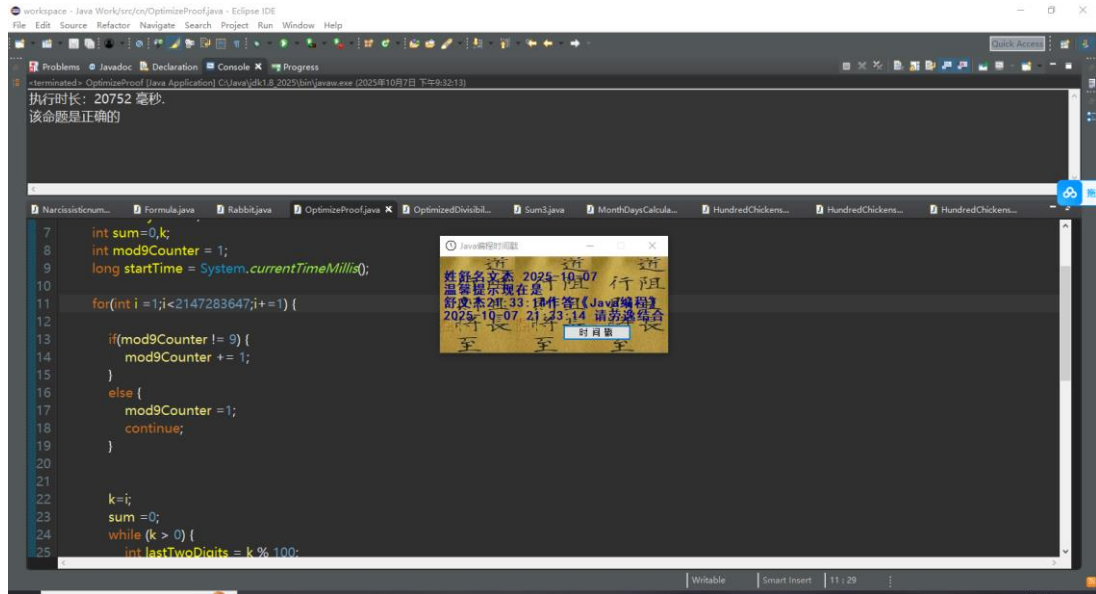


图 8 运行时间为 20 秒

但是，这还没有达到极限，我们还能进行优化，我们可以尽量减少除法的使用，我们将原来对于 9 的倍数的判定改为了计数器的形式，同时，我们将每位数两位两位进行分解，这样也可以减少除法次数。

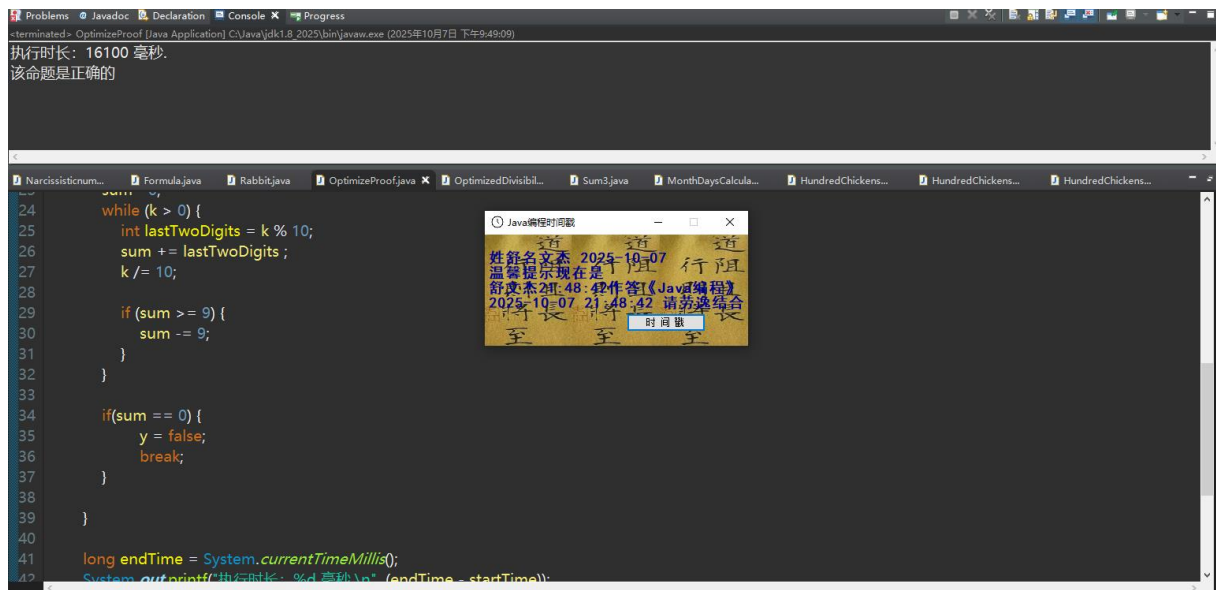


图 9 运行时间为 16 秒

可以发现，我们的优化效果斐然，我们的效率又提高了 20%，下一步，就是考虑我们的多线程如何操作来优化程序了。

详细学习过多线程后，我发现我需要创建一个线程池，同时将证明任务分割成线程的个数，同时让他们可以被中断（遇到反例停止），同时，我们需要建立一个可以被所有线程可见的原子变量以防止多线程的竞争问题，最后我需要建立 `CountDownLatch` 机制以协调多个线程间的进行（等待全部线程结束后再判断）。

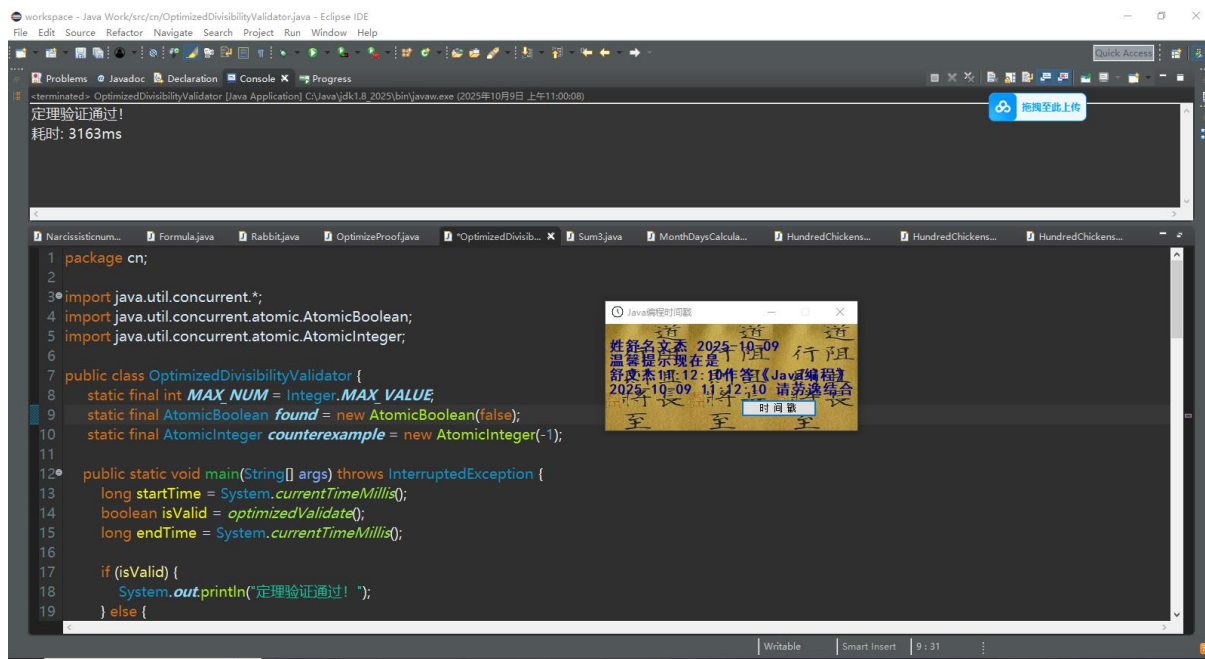


图 10 运行时间截图

可以发现，我们的证明完成成功，总共用时 3 秒多，这个程序的完成离不开互联网上开源大神的帮助，其中更有 Deepseek 带来的启发性思考，让我放弃 `Tread` 类而使用封装更加完善的 `Executorservice` (操作更为简单)，使我这个小白得以完成对这个证明的多线程优化。

6、有一对兔子，从出生后第 3 个月起每个月都生一对兔子，小兔子长到第三个月后每个月又生一对兔子，假如兔子都不死，问 2 年后的兔子总数为多少？

(1) 实验源程序

```
package cn;

public class Rabbit {
    static int rabbitNum(int k) {
        if(k==1) {
            return 4;
        }
        else if(k == 2) {
            return 6;
        }
    }
}
```

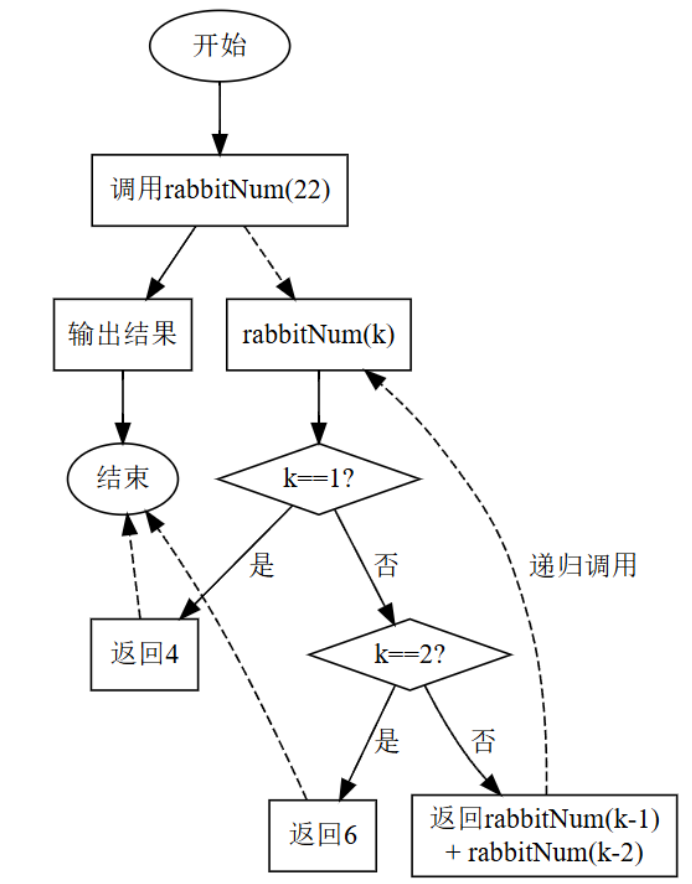
```

        else
            return rabbitNum(k-1) + rabbitNum(k-2);
    }

    public static void main(String[] args) {
        System.out.printf("两年的兔子总数为%d 只", rabbitNum(22));
    }
}

```

(2) 代码流程图



(3) 实验代码、过程、相应结果（截图）并对实验进行说明和分析：

实验代码思路：这是一个经典的斐波那契数列问题，同样的，它的递归式我们也可以以很简单的归纳出来，即 $n=1$ 时返回 4， $n=2$ 时返回 6，若不是一即返回 $f(n-2)+f(n-1)$ ，在继续枚举前 n 项阶乘后累加即可。

1) 编写 java 代码并运行

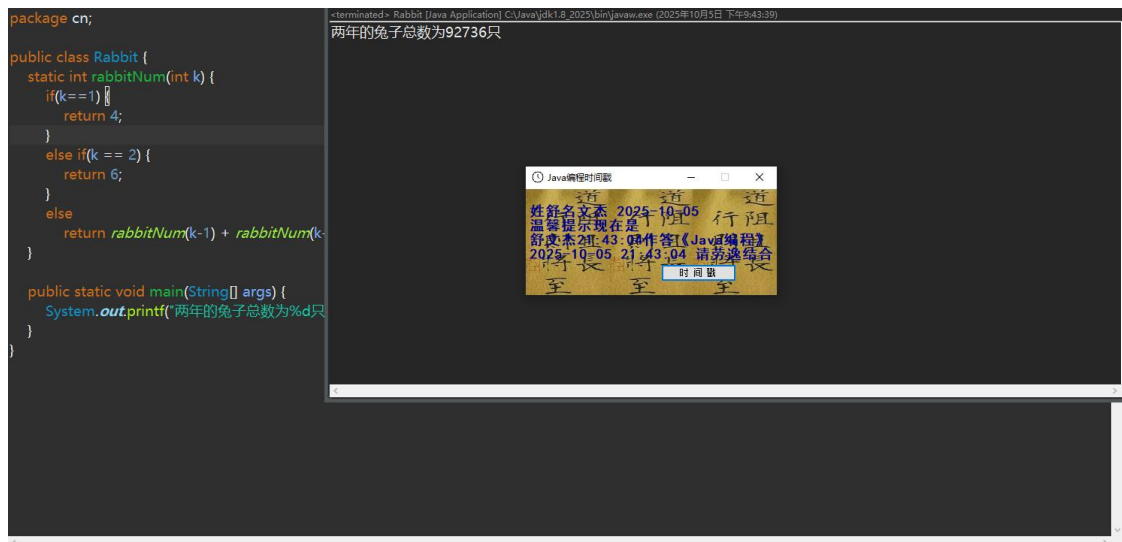


图 11 实验结果截图

可以说明我们的代码没有问题，实验成功。

【实验心得】

课堂问题回答：

1. 孙类能否继承爷爷类？

在 Java 中，孙类可以通过父类间接继承爷爷类的非私有成员和方法。Java 支持多层继承，孙类虽不能直接继承爷爷类，但能获取爷爷类中 public、protected 修饰的成员以及同包下默认权限的成员，而爷爷类的 private 成员无法被继承。同时，Java 只支持单继承，孙类不能同时直接继承父类和爷爷类。

2. 接口能继承吗？

Java 的接口可以继承接口，且与类的单继承不同，接口支持多继承，即一个接口可通过 extends 关键字同时继承多个父接口。子接口会继承父接口中的抽象方法、默认方法、静态方法和常量，实现子接口的类需实现所有继承而来的抽象方法。需注意，若父接口存在方法签名相同但返回值不同的抽象方法，子接口继承时会编译报错；子接口可重写父接口的默认方法，但不能重写静态方法。

3. Println 的母体是谁，为什么不在 System 类中？

我们常用的 println () 方法的“母体”是 java.io.PrintStream 类，它是该类的实例方法。日常通过 System.out.println () 调用时，System 是 java.lang 包下的系统类，其包含的静态成员变量 out 的类型为 PrintStream，out 作为 PrintStream 类型的对象，提供了对 println () 方法的调用入口。

在 System 类中查不到 println () 方法，是因为该方法并不属于 System 类。System 类的主要职责是管理系统资源，它通过静态成员变量 out（类型为 PrintStream）持有了能调用 println () 方法的对象；而 println () 方法实际定义在 PrintStream 类中，负责具体的打印逻辑。这种设计体现了职责分离，让 System 类专注于系统资源管理，PrintStream 类专注于打印功能实现。

本次实验感想：

本次实验，让我印象最深刻的就是优化了，从解方程的数学思想入手，我逐渐对于优化得心应手，一步步地进行优化给我带来了极大的成就感。

最开始写“百钱买百鸡”，我直接用了三重循环——母鸡、公鸡、小鸡挨个试，运行是能运行，但看着屏幕上“第 1 种买法、第 2 种买法”跳得有点慢，心里嘀咕“这代码是不是太‘笨’了？”。后来改成二重循环时，我盯着公式“ $chick = 100 - hen - cock$ ”琢磨了半天：原来可以用已知变量表示未知变量，少一层循环就能少很多次计算！再到一重循环时，推导“ $cock = (100 - 7 * hen) / 4$ ”，笔尖在草稿纸上算错两次才得到正确关系，最后运行时代码秒出结果，突然懂了“编程不是‘暴力试错’，而是‘用数学帮代码偷懒’”。

最让我印象深的是“验证 9 的倍数定理”的代码优化。第一次写的代码要跑 20 秒，盯着控制台的计时器，我想“能不能快点？”——先想到“9 的倍数不用验证，直接跳过”，改了 mod9Counter 计数器；又觉得“每次取一位数太费除法”，就改成“两位两位分解”（ $lastTwoDigits = k \% 100$ ），还加了“ $sum \geq 9$ 就减 9”减少计算量。当优化后的代码跑了 16 秒时，我盯着“执行时长：16000 毫秒”的提示，突然明白：“好代码不止是‘实现功能’，还要‘抠细节’——少一次除法、少一次循环，都是对效率的尊重”，这比单纯学会一个定理更有收获。到最后，我们再进行多线程优化，再此过程中，我对多线程并发计算有了更初步的理解。